

PCET'S

Pimpri Chinchwad College of Engineering, Pune

Name: Soham Kulkarni **PRN:** 124B1F056 **Batch:** TY - A3

Course: Design and Analysis of Algorithm Laboratory

ASSIGNMENT NO. 08

---

1. Problem Statement

SwiftShip delivers packages across multiple cities. A truck must start at a chosen city, visit every other city exactly once, and return to its starting point. Road distances, fuel consumption and travel times vary. Find a least-cost feasible tour while completing the trip within a specified time limit.

• Model the delivery network as a weighted, directed Traveling Salesperson Problem (TSP).

• Implement Least-Cost Branch and Bound, expanding the partial route with the smallest lower bound.

• Use varying fuel-consumption estimates to represent road conditions and enforce a total travel-time deadline.

• Prune unnecessary routes while retaining a proof of optimality for completed searches.

2. Course Objectives

• Apply Branch and Bound to a combinatorial optimization problem.

• Understand feasible upper bounds, admissible lower bounds, pruning and worst-case complexity.

3. Course Outcomes

Construct an exact time-constrained TSP model, implement a least-bound priority queue, recover the optimal tour, report infeasibility and assess scalability without assuming that all large instances are easy.

4. Model and Cost Calculation

Cities are vertices; an entered directed road gives its distance in whole kilometres, estimated fuel use in millilitres per kilometre, and travel time in whole minutes. Omitted roads are unavailable. To model an undirected connection, enter both directions. The truck uses one positive, whole-number fuel price in INR per litre.

```
fuel_mL(u,v) = distance_km(u,v) * rate_mL_per_km(u,v)
cost_milli_INR(u,v) = fuel_mL(u,v) * fuel_price_INR_per_L
```

Costs are stored as integer thousandths of an INR, avoiding floating-point pruning errors. Road-condition effects enter through the fuel rate and travel time. The objective is minimum total fuel cost, subject to the total travel-time limit; distance and duration are also reported. This is not simultaneous minimization of three objectives.

4.1. Branching and Feasible Upper Bounds

A node stores a partial city path, visited-city bit mask, incurred cost, elapsed time and cost lower bound. Branch by appending one unvisited city connected by an available road. Never revisit an intermediate city. Once all cities are present, explicitly test the return edge, including its cost and duration.

Two greedy trials, using low fuel cost and short time respectively, try to supply an initial feasible tour. A valid trial gives an upper bound. Greedy failure or a poor trial does not determine the answer; the exact Branch and Bound search still considers the remaining feasible possibilities.

4.2. Admissible Cost and Time Bounds

After fixing a partial path, each completion needs an outgoing edge from the current city and from every unvisited city. It also needs an incoming edge into every unvisited city and into the start. For these remaining rows and columns, ignore self-edges and unavailable roads; do not allow the current city to return to the start before all cities are visited.

```
outMin = sum of each remaining row minimum
inMin  = sum of each remaining column minimum
remainingCostBound = max(outMin, inMin)
nodeCostBound = incurredCost + remainingCostBound
```

Every tour uses one edge from each such row and column, so both sums separately underestimate or equal its remaining cost. Their maximum is still a lower bound. Do not add the two sums, since they bound the same edges. Ignoring assignment conflicts and subtours makes the bound weaker, not invalid. A missing required row or column proves that this branch cannot finish.

Apply the same calculation to travel-time weights. If elapsed time plus this remaining-time bound exceeds the deadline, the branch is infeasible. Separate minima for cost and time are safe bounds even when achieved by different relaxed routes.

5. Least-Cost Branch and Bound Algorithm

```
Try two greedy tours to initialize a feasible best cost
Build a root path containing only the start city
Compute root cost and time lower bounds
Insert root if it can improve the best cost and meet the deadline
While the min-priority queue is not empty:
    If its least bound cannot beat best cost, stop
    Remove the node with the least cost bound
    For each reachable unvisited next city:
        Extend the path, incurred cost and elapsed time
        Prune if prefix cost cannot improve or time already exceeds limit
        If every city is visited:
            Check return edge and total time
            Save this complete tour if it improves best cost
        Otherwise:
            Compute remaining cost and time lower bounds
            Prune if return completion is impossible
            Prune if deadline cannot be met or cost cannot improve
            Insert the remaining child into the priority queue
Print the proven optimum, or report no feasible tour
```

5.1. Why the Completed Search Is Exact

Branching covers all city visit orders from the fixed start. A branch is removed only when it cannot complete, cannot satisfy the deadline, or has a valid cost lower bound at least as high as a known feasible tour. Positive edge costs also make pruning by incurred cost safe. Therefore, no cheaper feasible tour is discarded. When the queue is empty or its smallest bound cannot improve the incumbent, that incumbent is optimal. If no feasible tour was found, the input has no feasible tour.

Least-Cost refers to the smallest **bound**, not simply the smallest cost paid so far. A min-priority queue implements this best-first selection. Equal-cost tours may return any optimum; there is no secondary distance or time tie objective.

5.2. Performance and Scalability

Brute force checks (N-1)! city orders from a fixed start. Branch and Bound can avoid many of them, but the worst case remains factorial. Each generated prefix needs O(N squared) bounding work and O(log(Q+1)) queue work, where Q is peak live frontier size. For G generated prefixes, a conservative search bound is O(G * (N squared + log(Q+1))). Memory is O(N squared + Q*N), including matrices and queued paths.

The program accepts 2-25 cities and uses 64-bit costs within explicit input limits. It has no heuristic early-stop rule. Hard 20+ city instances can still require substantial time and memory; no polynomial-time or universal fast-runtime guarantee is made. A structured 22-city test is supplied where the feasible seed equals the root bound, certifying the optimum without expanding a node. That is a property of the constructed instance, not a general benchmark.

6. Demo Input and Results

Use four cities and fuel price 100 INR/litre. The full directed-road input is in the separate demo guide. Every road has its own distance, fuel rate and duration. The following results include the final return trip.

Time limit	Optimal tour	Fuel cost	Travel time
90 min	1 - 3 - 2 - 4 - 1	550 INR	75 min
130 min	1 - 2 - 3 - 4 - 1	400 INR	120 min
70 min	No feasible tour	--	--

6.1. Output for a 90-Minute Limit

```
Optimal route: 1 -> 3 -> 2 -> 4 -> 1
Minimum fuel cost: 550.000 INR
Total distance: 55 km
Total fuel: 5.500 litres
Total travel time: 75 minutes
Time limit: 90 minutes
Optimality proven by LC Branch and Bound.
```

The first three roads have distance 15 km and rate 100 mL/km, consuming 1.5 litres each. The return road has distance 10 km and the same rate, consuming 1 litre. Total fuel is 5.5 litres, costing **550 INR**. Durations are 15 + 15 + 15 + 30 = **75 minutes**.

6.2. Effect of the Deadline

The cheaper 1 -> 2 -> 3 -> 4 -> 1 tour consumes 4 litres and costs 400 INR, but takes 120 minutes. It is invalid at the 90-minute deadline and becomes optimal when the limit is 130 minutes. No tour meets a 70-minute limit because the fastest complete tour takes 75 minutes.

A search may report zero expanded nodes if a feasible seed already matches the root lower bound. In that case, the lower bound proves that no cheaper route exists. Expanded-node, pruned-child and peak-queue statistics are printed after the result to illustrate the search effort.

6.3. Assumptions and Dynamic Conditions

All input road values are static during a solve. Update the input and rerun for a new road-condition snapshot. The model includes travel time only, with no service times, individual delivery windows, driver breaks or payload-dependent consumption. City IDs start at 1; rates, distances, prices and times use integer input units. Omitted roads, duplicate entries and invalid values are handled as described in the demo guide and README. All sample values are illustrative.

7. Conclusion

LC Branch and Bound finds an exact minimum-fuel-cost TSP tour within the total-time constraint. Admissible cost and time bounds prune unnecessary routes, while the priority queue focuses on the most promising lower bounds. Integer arithmetic preserves safe comparisons. Search effectiveness depends on the input, so worst-case factorial complexity remains a practical limitation.